INFORME GENERAL DE LABORATORIOS 01 A 04

Gestor de inventario persistente: excepciones, rutas, persistencia JSON, reportes CSV y arquitectura modular

1. Identificación

Campo	Dato
Aprendiz	Miller Joel Moncaleano Manjarres
Ficha	3388087
Programa de formación	Análisis y Desarrollo de Software (SENA)
Nombre del laboratorio	Laboratorios 01 a 04: fundamentos de Python, estructuras de datos, POO, persistencia en archivos (JSON / CSV) y robustez
Repositorio personal (GitHub)	https://github.com/Joel90900/mi-repositorio-python
Entorno de ejecución	Python 3.14, Windows, VS Code / Terminal PowerShell

El repositorio contiene los ejercicios y proyectos de los cuatro laboratorios, con un commit por laboratorio.

2. Estado del proyecto

2.1 Funcionalidades terminadas

- **Práctica 4.1 (excepciones y validaciones):** conversiones seguras con try, except, else y finally; captura de ValueError, TypeError, ZeroDivisionError y FileNotFoundError; uso de raise ValueError para reglas del dominio.
- **Práctica 4.2 (archivos de texto y rutas con pathlib):** lectura y escritura con encoding utf-8; context manager with open; diferencia entre modos w, a y r; rutas anclada con Path(__file__).resolve().parent.
- **Práctica 4.3 (persistencia JSON y CSV):** serialización con json.dump y ensure_ascii=False; métodos a_diccionario y desde_diccionario; captura de json.JSONDecodeError; exportación e importación con csv.DictWriter y csv.DictReader.
- **Práctica 4.4 (POO intermedia):** herencia con super().__init__; sobrescritura de enviar() y describir(); polimorfismo sin if/elif; composición (el Inventario contiene Producto y usa un motor de almacenamiento).
- **Práctica 4.5 (gestor de inventario persistente):** producto.py (validación estricta, con math.isfinite), inventario.py (colecciones, duplicados, resúmenes y carga atómica), almacenamiento.py (tres estados del archivo, guardado con .tmp y respaldo .bak), reportes.py (CSV con valor_inventario = precio x cantidad) y main.py (menú de 10 opciones con reintentos).

2.2 Dificultades y soluciones técnicas

- **Valores especiales en punto flotante (nan, inf):** float("inf") > 0 evalúa como verdadero. Se añadió math.isfinite en Producto y en leer_precio().
- **Sobrescritura destructiva de JSON dañado:** si json.load() fallaba y se asignaba datos = [], el siguiente guardado borraba los datos recuperables. Se resolvió con la bandera persistencia_habilitada, que bloquea el guardado al detectar JSONDecodeError.
- **Rutas relativas:** ejecutar python main.py desde otra carpeta buscaba los archivos en lugares incorrectos. Se corrigió anclando las rutas con BASE_DIR = Path(__file__).resolve().parent.

2.3 Pendientes

Las prácticas 4.1 a 4.5 del laboratorio 04 están terminadas.

2.4 Limitaciones conocidas

- La persistencia usa un único archivo JSON local; no hay control de concurrencia si dos procesos escriben a la vez.

- El respaldo .bak guarda solo la copia previa al último guardado; no es un historial de versiones.
- La aplicación es de consola y de un solo usuario.
- Las pruebas están en un script propio (test_gestor.py), no en un framework como pytest. La prueba de archivo dañado se hizo de forma manual.

3. Seguimiento del desarrollo y trazabilidad (commits)

El historial tiene un commit por laboratorio, en el orden en que se hizo el trabajo. Los hashes corresponden a la salida de `git log --oneline` del repositorio.

Commit	Laboratorio	Cambio realizado	Razón del cambio	Comprobación y resultado
eb71e5d	01	feat(lab01): fundamentos de Python. Variables, operaciones, entrada con input() y error_varibles.py.	Organizar los ejercicios de fundamentos en su propio commit como punto de partida.	Ejecución de cada script en consola.
2e9cf66	02	feat(lab02): estructuras de datos y control. Listas, tuplas, diccionarios, ciclos, menú y gestor de inventario.	Introducir colecciones y menús interactivos.	Ejecución de los scripts de ciclos, colecciones y del menú en consola.
6803efc	03	feat(lab03): POO y modularidad. Clases Producto e Inventario y separación en módulos.	Abstraer el inventario en clases y separar la interfaz de la lógica.	Ejecución del programa de inventario en memoria.
2040fca	04	feat(lab04): persistencia, excepciones y gestor de inventario. Prácticas 4.1 a 4.5.	Evitar la pérdida de datos al cerrar, proteger el JSON dañado y exportar reportes.	Sección 4 de este informe: guardado y recuperación, JSON dañado y reporte CSV.

4. Evidencias de comprobación

4.1 Comprobación numérica con los productos de la guía

Productos registrados: P001 Teclado ($85.000, 4 unidades, Periféricos), P002 Mouse ($45.000, 0 unidades, Periféricos) y P003 Monitor ($700.000, 2 unidades, Componentes).

Prueba	Resultado esperado	Resultado obtenido
Resumen inicial (opción 7)	3 productos, 6 unidades, valor $1.740.000, sin existencias: Mouse	Igual al esperado
Actualizar Mouse de 0 a 5 y consultar	11 unidades, valor $1.965.000	Igual al esperado
Cerrar (opción 0), reiniciar y consultar	Se recuperan los 3 productos, 11 unidades y $1.965.000	Datos recuperados completos
Reporte CSV: valor_inventario de P002	225000.00 (45000 x 5)	225000.00

Contenido de reportes/inventario.csv:

```
codigo,nombre,categoria,precio,cantidad,valor_inventario
P001,Teclado,Periféricos,85000.00,4,340000.00
P002,Mouse,Periféricos,45000.00,5,225000.00
P003,Monitor,Componentes,700000.00,2,1400000.00
```

4.2 Prueba de archivo dañado (corrupción controlada)

- Con un inventario válido guardado en disco, se editó una copia del archivo eliminando la coma entre dos objetos JSON.
- Al iniciar la aplicación, almacenamiento.py capturó json.JSONDecodeError e indicó la línea y la columna del error.

- El programa mostró: "ERROR CRÍTICO: El archivo 'productos.json' tiene un error de formato JSON... guardado automático DESHABILITADO".
- En disco, el archivo conservó su texto original y no fue reemplazado por [].
- Se corrigió la sintaxis, se reinició la aplicación y los productos se recuperaron por completo.

5. Comparación de avances entre laboratorios

Laboratorio	Avance principal	Limitación que motivó el siguiente paso
01: Fundamentos	Variables, operaciones aritméticas y entrada con input().	Scripts lineales, sin modularización; todo el cálculo en un solo archivo.
02: Estructuras y control	Listas, tuplas y diccionarios; ciclos while y for; menú de consola.	Los datos se manejaban con variables sueltas y sin una abstracción propia.
03: POO y modularidad	Clases Producto e Inventario; encapsulamiento con @property; separación entre interfaz y lógica.	Los datos se perdían al cerrar el programa.
04: Persistencia y robustez	Excepciones específicas; rutas con pathlib; JSON con guardado atómico; CSV; herencia y composición; protección ante datos dañados.	Versión actual. Separa objetos de negocio (que no conocen el disco) de adaptadores de infraestructura (almacenamiento y reportes).

En el paso del laboratorio 03 al 04 el programa dejó de perder datos al cerrarse. Además, la validación se trasladó a las clases de dominio, y los cambios de almacenamiento y reportes quedaron aislados en sus propios módulos.

6. Análisis situacional (cinco preguntas)

Pregunta 1. ¿Qué harías si el programa se cierra inesperadamente y, al volver a ejecutarlo, los productos ya no aparecen?

- **Qué haría:** comprobar si los cambios llegaron a disco revisando la fecha de modificación y el contenido de datos/productos.json. Si el archivo existe pero no tiene los registros, la operación se aceptó en memoria pero no se sincronizó con el disco.
- **Rutas:** revisar si el programa usa una ruta relativa como Path("datos/productos.json"). En ese caso el archivo se crea según el directorio desde el que se ejecute. La solución es anclar la ruta con Path(__file__).resolve().parent (commit del laboratorio 04).
- **Cómo lo implementaría:** guardado automático tras cada registrar, actualizar o eliminar exitoso, con try/except OSError para avisar al usuario si falla la escritura.
- **Cómo lo comprobaría:** en test_gestor.py, registrar un producto, llamar a guardar_inventario(), eliminar la instancia con del, reconstruir con cargar_inventario() y verificar con aserciones que los atributos coinciden.

Pregunta 2. ¿Qué harías si productos.json existe, pero tiene una coma faltante y el programa no puede leerlo?

- **Cómo identificaría el error:** json.load() lanza json.JSONDecodeError, que indica la línea (lineno), la columna (colno) y la descripción (msg).
- **Por qué no reemplazarlo por uno vacío:** si se capturara un Exception genérico y se ejecutara datos = [] seguido de un guardado, se borrarían cientos de productos que solo necesitan una coma para recuperarse.
- **Procedimiento de recuperación:** almacenamiento.py muestra el error con línea y columna, desactiva persistencia_habilitada y no toca el disco. Luego se abre productos.json (o el respaldo productos.json.bak), se corrige la coma y se reinicia el programa (commit del laboratorio 04).
- **Cómo lo comprobaría:** la prueba manual de la sección 4.2: el archivo dañado debe quedar idéntico tras intentar un guardado.

Pregunta 3. ¿Qué harías si el inventario guarda correctamente un producto, pero al recuperar el JSON aparecen dos registros con el mismo código?

- **Dónde comprobar la unicidad:** en dos niveles. En inventario.registrar_producto(), antes de agregar el producto a la lista interna; y en Inventario.desde_lista_datos(), al cargar desde el archivo.
- **Cómo evitar una carga parcial:** se usa un patrón de construcción atómica. Un conjunto local codigos_vistos y un inventario temporal reciben los productos. Si un código ya existe, se lanza ValueError y el temporal se descarta. Así, si el registro 7 de 10 está repetido, los primeros 6 nunca llegan al inventario activo.
- **Cómo lo comprobaría:** crear un JSON de prueba con dos registros con código P001. Llamar a cargar_inventario() sobre ese archivo, verificar que se captura ValueError y que el inventario activo queda sin cargas incompletas.

Pregunta 4. ¿Qué harías si un reporte CSV muestra el precio unitario en la columna del valor total, aunque cada producto tenga varias unidades?

- **Qué revisaría:** el módulo reportes.py y el diccionario que alimenta csv.DictWriter. Si valor_inventario recibe p.precio, la asignación es incorrecta. La fórmula de negocio es valor_inventario = precio x cantidad.
- **Dónde debe hacerse el cálculo:** en el método calcular_valor() de la clase Producto, que es quien conoce sus datos. reportes.py solo lo consume al armar cada fila.
- **Cómo lo comprobaría:** con el producto de precio 45000 y cantidad 5, la columna debe contener 225000.00. En la suite automatizada, leer el CSV con csv.DictReader y verificar que float(fila["valor_inventario"]) == float(fila["precio"]) * int(fila["cantidad"]).

Pregunta 5. ¿Qué harías si, tras separar el programa en clases y módulos, una corrección al guardado hace que el inventario deje de cargar datos?

- **Historial de Git:** ejecutar git log --oneline -n 10 y git log -p almacenamiento.py para ver los cambios recientes en la persistencia. Con git diff <hash_anterior> HEAD se comparan las líneas modificadas en guardar_inventario(). Si la causa no es clara, git bisect aísla el commit que introdujo la regresión, marcando el actual como bad y uno anterior funcional como good.
- **Reproducción:** comprobar si la corrección cambió las claves de a_diccionario (por ejemplo, "codigo" por "cod"). Esto provoca un KeyError en desde_diccionario(). Un script aislado que guarde y cargue inmediatamente reproduce el fallo.
- **Corrección y evidencia:** restaurar la paridad de claves entre a_diccionario() y desde_diccionario(), ejecutar test_gestor.py completo y registrar el commit correctivo con un mensaje convencional, por ejemplo: fix(almacenamiento): corregir discrepancia de claves en serialización que impedía la carga.